

ДЗ 1 БД

Бизнес-домен:

Вы разрабатываете базу данных для платформы онлайн-образования. Платформа позволяет пользователям регистрироваться, выбирать курсы и проходить обучение.

Каждый курс состоит из нескольких уроков. Преподаватели могут создавать курсы и получать отчисления от продаж. Студенты могут оставлять отзывы на курсы и оценивать их по шкале от 1 до 5.

Задание 1. Концептуальная модель

6 основных сущностей

- User
- Course
- Lesson
- Enrollment
- Review
- Payout

Обоснование

User(пользователь)

Бизнес-требование: *«Пользователи регистрируются»*

Обоснование выбора: Данная сущность необходима, чтобы знать кто заходит на сайт. Отдельная таблица нужна, чтобы человек мог один раз зарегистрироваться и войти под своим логином и паролем. Без этого мы не поймем, кто из них студент, а кто преподаватель.

Course(курс)

Бизнес-требование: *«Создание курсов преподавателями для студентов»*

Обоснование выбора: Курс — это главный «товар» на сайте. Отдельная таблица нужна, чтобы хранить его название, описание и цену в одном месте. Если что-то в описании поменяется, мы исправим это в одной строчке, а не во всей базе.

Lesson(урок)

Бизнес-требование: *«Каждый курс состоит из уроков»*

Обоснование выбора: Курс состоит из нескольких частей, поэтому его нельзя записать в одну строку. Мы выносим уроки в отдельную таблицу, чтобы для каждого хранить его номер по порядку, тему и ссылку на видео.

Enrollment(запись на курс)

Бизнес-требование: «Выбор и прохождение курсов»

Обоснование выбора: Эта таблица связывает студента с курсом и запоминает: когда он купил курс, сколько заплатил и прошел ли его до конца.

Review(отзыв)

Бизнес-требование: «Студенты могут оставлять отзывы на курсы и оценивать их по шкале от 1 до 5»

Обоснование выбора: Нам нужна отдельная таблица, чтобы сохранять текст отзыва и оценку от студента. Это позволяет показывать отзывы на странице курса, считать его средний рейтинг и следить, чтобы один студент не оставлял больше одного отзыва.

Payout(выплата преподавателю)

Бизнес-требование: «Преподаватели могут создавать курсы и получать отчисления от продаж»

Обоснование выбора: Курс за месяц может купить много студентов, но платформа не переводит преподавателю деньги за каждую продажу отдельно — она копит их и платит одной суммой раз в период (например, раз в месяц). Отдельная таблица нужна, чтобы зафиксировать сам факт и дату такой выплаты: без неё непонятно, какая сумма уже перечислена преподавателю, а какая только начислена, но ещё не выплачена.

Связи между ними

USER 1 – N Course (Преподаватель создает курсы)

User (1) – (N) Enrollment (Студент может записаться на несколько курсов)

User (1) – (N) Review (Студент оставляет отзывы на курсы)

User (1) – (N) Payout (Преподаватель получает несколько выплат за разные периоды)

Course (1) – (N) Lesson (Курс содержит несколько уроков)

Course (1) – Review (N) (На один курс может быть много отзывов)

Course (1) – (N) Enrollment (На один курс может быть много записей)

Задание 2. Логическая модель

Описание сущностей и их атрибутов

Как бизнес-требования повлияли на выбор атрибутов: users.role появился из требования «нужно разграничивать студентов и преподавателей»; courses.price и courses.commission_rate — из требования «преподаватели получают отчисления от продаж» (нужна сумма, от которой считать выплату, и доля, которую забирает платформа); lessons.order_number — из требования «курс состоит из нескольких уроков», которые должны идти по порядку; enrollments.enrolled_at, enrollments.amount_paid, enrollments.payment_status и enrollments.status — из требования «студенты выбирают и

проходят курсы» (нужно зафиксировать сам факт оплаты, сколько реально заплачено и статус прохождения — это разные вещи); reviews.rating и comment — напрямую из требования «студенты оставляют отзывы и оценивают курсы по шкале от 1 до 5»; payouts (вся сущность) — из требования «преподаватели получают отчисления от продаж», так как выплата — это отдельное событие со своей датой и статусом, а не просто сумма, посчитанная «на лету».

User

- **user_id** (INTEGER, Primary Key) — Уникальный идентификатор пользователя.
- **name** (VARCHAR(100)) — Имя и фамилия пользователя.
- **email** (VARCHAR(255), Unique, Not Null) — Электронная почта.
- **password** (VARCHAR(255), Not Null) — Хэш пароля пользователя.
- **role** (VARCHAR(20), Not Null) — Роль в системе ('student' или 'teacher') для разграничения прав.

Course

- **course_id** (INTEGER, Primary Key) — Уникальный идентификатор курса.
- **teacher_id** (INTEGER, Foreign Key → User.user_id) — Ссылка на преподавателя, создавшего курс.
- **name_course** (VARCHAR(200), Not Null) — Название курса.
- **price** (DECIMAL(10, 2), Not Null) — Стоимость курса.
- **commission_rate** (DECIMAL(5, 2), Not Null, Default 30.00) — Процент, который платформа удерживает себе с каждой продажи этого курса; доля преподавателя считается как $(100 - \text{commission_rate})\%$.

Lesson

- **lesson_id** (INTEGER, Primary Key) — Уникальный идентификатор урока.
- **course_id** (INTEGER, Foreign Key → Course.course_id) — Ссылка на курс, к которому относится урок.
- **name_lesson** (VARCHAR(200), Not Null) — Название урока.
- **lesson_time** (INTEGER) — Длительность урока (в минутах).
- **order_number** (INTEGER, Not Null) — Порядковый номер урока внутри курса.

Enrollment

- **enrollment_id** (INTEGER, Primary Key) — Уникальный номер записи на курс.
- **user_id** (INTEGER, Foreign Key → User.user_id) — Ссылка на записавшегося студента.
- **course_id** (INTEGER, Foreign Key → Course.course_id) — Ссылка на купленный курс.

- **enrolled_at** (TIMESTAMP, Not Null) — Дата и время записи на курс.
- **amount_paid** (DECIMAL(10, 2), Not Null) — Сумма, которую студент реально заплатил за курс на момент покупки (фиксируется отдельно от текущей цены курса, чтобы более позднее изменение course.price не искажало историю платежей).
- **payment_status** (VARCHAR(20), Not Null, Default 'paid') — Статус оплаты ('paid', 'refunded'), отдельно от статуса прохождения курса — студент мог оплатить, но не начать проходить, либо пройти курс, а затем оформить возврат.
- **status** (VARCHAR(20), Not Null) — Статус прохождения ('active', 'completed').

Review

- **review_id** (INTEGER, Primary Key) — Уникальный идентификатор отзыва.
- **user_id** (INTEGER, Foreign Key → User.user_id) — Ссылка на автора отзыва (студента).
- **course_id** (INTEGER, Foreign Key → Course.course_id) — Ссылка на оцениваемый курс.
- **rating** (INTEGER, Not Null) — Оценка курса по шкале от 1 до 5.
- **comment** (TEXT) — Текст отзыва.
- **created_at** (TIMESTAMP, Not Null) — Дата публикации отзыва.

Payout

- **payout_id** (INTEGER, Primary Key) — Уникальный идентификатор выплаты.
- **teacher_id** (INTEGER, Foreign Key → User.user_id) — Ссылка на преподавателя, которому начислена выплата.
- **period_start** (DATE, Not Null) — Начало периода, за который считается выплата.
- **period_end** (DATE, Not Null) — Конец периода, за который считается выплата.
- **amount** (DECIMAL(10, 2), Not Null) — Итоговая сумма к выплате преподавателю за период (сумма всех amount_paid по его курсам за период за вычетом commission_rate).
- **payout_status** (VARCHAR(20), Not Null, Default 'pending') — Статус выплаты ('pending', 'paid').
- **paid_at** (TIMESTAMP) — Дата фактической выплаты; пусто, пока выплата не проведена.

Кардинальность связей

USER 1 – N Course

Со стороны User: Один преподаватель может создать 0 или несколько курсов (1 к N).

Со стороны Course: Каждый курс обязательно создается строго одним преподавателем (N к 1).

User (1) – (N) Enrollment

Со стороны User: Студент может сделать 0 или несколько записей на курсы (1 к N).

Со стороны Enrollment: Каждая запись принадлежит строго одному пользователю (N к 1).

User (1) – (N) Review

Со стороны User: Пользователь может написать от 0 до нескольких отзывов на разные курсы (1 к N).

Со стороны Review: Каждый отзыв принадлежит конкретному пользователю.

User (1) – (N) Payout

Со стороны User: Преподаватель может иметь 0 или несколько выплат за разные периоды (1 к N).

Со стороны Payout: Каждая выплата относится строго к одному преподавателю (N к 1).

Course (1) – (N) Lesson

Со стороны Course: Курс может содержать 0 или несколько уроков (1 к N).

Со стороны Lesson: Урок принадлежит строго одному конкретному курсу (N к 1).

Course (1) – Review (N)

Со стороны Course: Курс может иметь от 0 до множества отзывов (1 к N).

Со стороны Review: Каждый отзыв относится к конкретному курсу.

Course (1) – (N) Enrollment

Со стороны Course: На курс может записаться 0 или много студентов (1 к N).

Со стороны Enrollment: Каждая запись относится строго к одному курсу (N к 1).

Неидентифицирующая

Классификация связей

Примечание: связь классифицируется как идентифицирующая по смысловой (бизнес) зависимости — существует ли дочерняя запись без родителя, — а не по структуре первичного ключа. На уровне схемы все таблицы используют собственные суррогатные SERIAL-ключи, поэтому формально (по составу PK) все связи в физической модели неидентифицирующие.

User (teacher) → Course

Неидентифицирующая

Курс — самостоятельная сущность: у него своё название, цена, содержание. Смысл курса не теряется, если его переназначить другому преподавателю — курс не перестаёт быть тем же курсом.

Course → Lesson

Идентифицирующая

Урок вне курса не имеет смысла — это часть курса, а не отдельный объект.

User (student) → Enrollment

Идентифицирующая

Enrollment — это когда конкретный студент записался на конкретный курс. Без студента запись не существует и не имеет смысла сама по себе.

User (student) → Review

Идентифицирующая

Отзыв не существует без автора — это оценка конкретного студента.

Course → Review

Идентифицирующая

Отзыв не существует без курса, к которому он относится.

Course → Enrollment

Неидентифицирующая

Enrollment имеет собственный суррогатный ключ `enrollment_id`, не включающий `course_id` в свой состав, поэтому формально связь неидентифицирующая — хотя семантически запись без курса не имеет смысла.

User (teacher) → Payout

Неидентифицирующая

Payout имеет собственный суррогатный ключ `payout_id`, не включающий `teacher_id` в свой состав, поэтому формально связь неидентифицирующая — хотя семантически выплата без преподавателя не имеет смысла (та же логика, что и у Course → Enrollment).

Задание 3. Физическая модель

Как бизнес-требования повлияли на выбор ограничений

- `users.role`: NOT NULL + CHECK (`role IN ('student', 'teacher')`) — по требованию «нужно разграничивать студентов и преподавателей» это должно проверяться на уровне базы.
- `users.email`: UNIQUE NOT NULL — при регистрации один email соответствует ровно одному аккаунту.

- `courses.price`: тип `DECIMAL(10, 2)` для точных денежных сумм (не `FLOAT`, который округляет неточно) и `CHECK (price >= 0)`, так как курс платный и отрицательная цена невозможна.
- `courses.commission_rate`: `NOT NULL DEFAULT 30.00 + CHECK (commission_rate BETWEEN 0 AND 100)` — по требованию «преподаватели получают отчисления от продаж» комиссия платформы обязана быть процентом (0–100), а значение по умолчанию задаёт стандартную ставку, если для курса не указана особая.
- `lessons.order_number`: `CHECK (order_number > 0)` и `UNIQUE (course_id, order_number)` — по требованию «уроки идут по порядку внутри курса» два урока одного курса не могут иметь одинаковый номер (та же идея, что составной `PRIMARY KEY` в `task_tags` из семинара — уникальна не колонка сама по себе, а пара колонок).
- `enrollments.amount_paid`: `NOT NULL + CHECK (amount_paid >= 0)` — фиксирует, сколько студент реально заплатил в момент покупки, отдельно от текущей цены курса; сумма не может быть отрицательной.
- `enrollments.payment_status`: `CHECK (payment_status IN ('paid', 'refunded'))` — отражает статус оплаты отдельно от статуса прохождения курса.
- `enrollments.status`: `CHECK (status IN ('active', 'completed'))` — отражает статусы прохождения курса.
- `enrollments`: `UNIQUE (user_id, course_id)` — повторная запись студента на тот же курс не нужна.
- `reviews.rating`: `CHECK (rating BETWEEN 1 AND 5)` — прямое отражение требования «оценивать курсы по шкале от 1 до 5».
- `reviews`: `UNIQUE (user_id, course_id)` — один студент не должен оставлять больше одного отзыва на курс.
- `payouts.amount`: `NOT NULL + CHECK (amount >= 0)` — итоговая сумма выплаты не может быть отрицательной.
- `payouts`: `CHECK (period_end >= period_start)` — период выплаты должен быть корректным (конец не раньше начала), по аналогии с `CHECK (deadline >= start_date)` из семинара.
- `payouts.payout_status`: `NOT NULL DEFAULT 'pending' + CHECK (payout_status IN ('pending', 'paid'))` — отражает, начислена выплата или уже фактически переведена преподавателю.
- Внешние ключи: `courses → users` и `payouts → users` — `ON DELETE RESTRICT` (неидентифицирующая связь: при удалении преподавателя курсы, история продаж и история выплат не должны пропадать, как в семинаре с `changed_by_employee_id → employees`). Остальные связи (`lessons → courses`, `enrollments → users/courses`,

reviews → users/courses) — ON DELETE CASCADE, так как в Задании 2 они отмечены как идентифицирующие: дочерняя запись не имеет смысла без родителя (как projects → tasks в семинаре).

- На все внешние ключи добавлены индексы (CREATE INDEX), так как по частым запросам (Задание 4) регулярно требуется выборка по курсу/студенту/преподавателю — без индекса такие запросы будут сканировать всю таблицу.

SQL-скрипт (PostgreSQL)

```
DROP TABLE IF EXISTS payouts CASCADE;  
DROP TABLE IF EXISTS reviews CASCADE;  
DROP TABLE IF EXISTS enrollments CASCADE;  
DROP TABLE IF EXISTS lessons CASCADE;  
DROP TABLE IF EXISTS courses CASCADE;  
DROP TABLE IF EXISTS users CASCADE;
```

```
CREATE TABLE users (  
    user_id SERIAL PRIMARY KEY,  
    name VARCHAR(100) NOT NULL,  
    email VARCHAR(255) UNIQUE NOT NULL,  
    password VARCHAR(255) NOT NULL,  
    role VARCHAR(20) NOT NULL  
        CHECK (role IN ('student', 'teacher'))  
);
```

```
CREATE TABLE courses (  
    course_id SERIAL PRIMARY KEY,  
    teacher_id INTEGER NOT NULL,  
    name_course VARCHAR(200) NOT NULL,  
    price DECIMAL(10, 2) NOT NULL CHECK (price >= 0),  
    commission_rate DECIMAL(5, 2) NOT NULL DEFAULT 30.00  
        CHECK (commission_rate BETWEEN 0 AND 100),  
    FOREIGN KEY (teacher_id) REFERENCES users(user_id) ON DELETE RESTRICT  
);
```

```
CREATE INDEX idx_courses_teacher_id ON courses(teacher_id);
```

```
CREATE TABLE lessons (  
    lesson_id SERIAL PRIMARY KEY,  
    course_id INTEGER NOT NULL,  
    name_lesson VARCHAR(200) NOT NULL,
```

```
lesson_time INTEGER CHECK (lesson_time > 0),  
order_number INTEGER NOT NULL CHECK (order_number > 0),  
FOREIGN KEY (course_id) REFERENCES courses(course_id) ON DELETE CASCADE,  
UNIQUE (course_id, order_number)  
);
```

```
CREATE INDEX idx_lessons_course_id ON lessons(course_id);
```

```
CREATE TABLE enrollments (  
  enrollment_id SERIAL PRIMARY KEY,  
  user_id INTEGER NOT NULL,  
  course_id INTEGER NOT NULL,  
  enrolled_at TIMESTAMP DEFAULT NOW(),  
  amount_paid DECIMAL(10, 2) NOT NULL CHECK (amount_paid >= 0),  
  payment_status VARCHAR(20) NOT NULL DEFAULT 'paid'  
    CHECK (payment_status IN ('paid', 'refunded')),  
  status VARCHAR(20) DEFAULT 'active'  
    CHECK (status IN ('active', 'completed')),  
  FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE,  
  FOREIGN KEY (course_id) REFERENCES courses(course_id) ON DELETE CASCADE,  
  UNIQUE (user_id, course_id)  
);
```

```
CREATE INDEX idx_enrollments_user_id ON enrollments(user_id);  
CREATE INDEX idx_enrollments_course_id ON enrollments(course_id);
```

```
CREATE TABLE reviews (  
  review_id SERIAL PRIMARY KEY,  
  user_id INTEGER NOT NULL,  
  course_id INTEGER NOT NULL,  
  rating INTEGER NOT NULL CHECK (rating BETWEEN 1 AND 5),  
  comment TEXT,  
  created_at TIMESTAMP DEFAULT NOW(),  
  FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE,  
  FOREIGN KEY (course_id) REFERENCES courses(course_id) ON DELETE CASCADE,  
  UNIQUE (user_id, course_id)  
);
```

```
CREATE INDEX idx_reviews_user_id ON reviews(user_id);  
CREATE INDEX idx_reviews_course_id ON reviews(course_id);
```

```

CREATE TABLE payouts (
  payout_id SERIAL PRIMARY KEY,
  teacher_id INTEGER NOT NULL,
  period_start DATE NOT NULL,
  period_end DATE NOT NULL,
  amount DECIMAL(10, 2) NOT NULL CHECK (amount >= 0),
  payout_status VARCHAR(20) NOT NULL DEFAULT 'pending'
    CHECK (payout_status IN ('pending', 'paid')),
  paid_at TIMESTAMP,
  FOREIGN KEY (teacher_id) REFERENCES users(user_id) ON DELETE RESTRICT,
  CHECK (period_end >= period_start)
);

```

```

CREATE INDEX idx_payouts_teacher_id ON payouts(teacher_id);

```

Задание 4. Частые запросы

1. «Вывести список всех курсов с именем преподавателя, ценой и общее количество записавшихся студентов».
2. «Найти top-5 самых популярных курсов по средней оценке и количеству отзывов».
3. «Показать все уроки конкретного курса в правильном порядке их прохождения».
4. «Получить историю обучения конкретного студента: список купленных курсов, статус прохождения и оставленные отзывы».
5. «Рассчитать общую сумму отчислений за текущий месяц для конкретного преподавателя по всем его созданным курсам» — считается как сумма $\text{enrollments.amount_paid} \times (1 - \text{commission_rate} / 100)$ по всем курсам преподавателя за месяц, либо берётся напрямую из уже созданной записи в payouts за этот период, если выплата уже начислена.